2024W 052600-1 Signal and Image Processing

Assignment 3

Submission deadline: Friday, January 17th, 2025, 23:59

For this assignment, please submit **a Jupyter Notebook** with Python (v $\geq$ 3.8) code and markdown comments containing the answers to the questions. **Do not use any off-the-shelf libraries or functions** in your solution code unless we explicitly mention it. Please ask if you want to use some functions not mentioned here; otherwise, you will lose all points for that question due to the unpermitted usage of a function or library.

Allowed libraries/functions: basic **`numpy`** (e.g., `cos`, `sin`, `ones`, `zeros`, `arrange`, `linspace`, `sqrt`, `abs`, `max`, etc.) and **`matplotlib.pyplot`** functions.

In all frequency-domain plots, the x-axis should display the frequency in Hz, and in all time-domain plots, the x-axis should display the time in seconds. Make sure to put the frequency 0Hz and the time 0s in the center of your plots when your data has entries for negative frequencies. Shift your arrays using `numpy.fft.fftshift`.

Please name your file **`SIP24W_A3_LastName_StudentID.ipynb`** and include your name and student ID at the beginning of the notebook. If the direct upload of the notebook is not possible, you may upload a zipped file instead.

1 Plotting signals [8 P]

In this exercise, you will define auxiliary functions to plot signals in the time and frequency domains. You are allowed to reuse these to plot signals in other exercises.

(a) Time domain [2 P]

Define a function `plot_time(x, fs)` to plot a signal in the time domain, where x is the signal (real-valued numpy array of arbitrary length), and fs is the sampling frequency in Hz (float). The x-axis should display the time in seconds and should be labeled accordingly.

(b) Frequency domain [4 P]

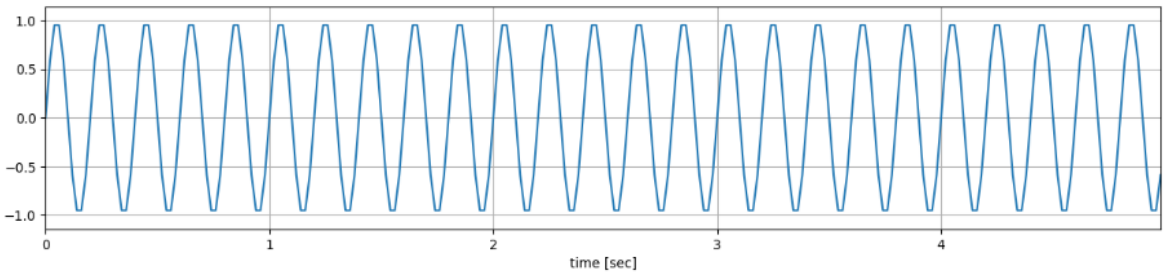
Define a function `plot_freq(X, fs)` to plot a signal in the frequency domain, where X is the signal in the frequency domain (complex-valued numpy array of arbitrary length), and fs is the sampling frequency in Hz (float). The function should plot two side-by-side plots, showing the amplitude spectrum on the left and the phase spectrum on the right. The x-axis should display the frequency in Hz, with 0Hz in the center of the plot. Make sure that your function is able to correctly plot signals of different lengths and sampling frequencies!

Hint: Use `numpy.fft.fftshift` to shift the arrays obtained by `numpy.fft.fft`.

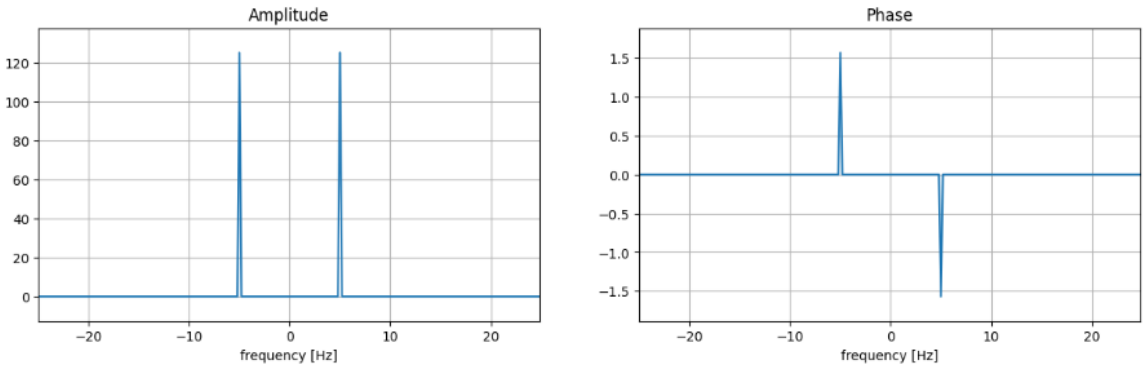
(c) Test the functions [2 P]

Define a sine signal with a frequency of 5Hz, with a length of 5 seconds, and with a sampling frequency of 50Hz. Compute its DFT using `numpy.fft.fft`. Plot the signal in the time and frequency domain using your previously defined functions. The results should look like this:

```
In [7]: plot_time(x, fs)
```




```
In [8]: plot_freq(X, fs)
```



2 Multitaper spectral analysis [21 P]

In this exercise, you will implement a function for multitaper spectral analysis, a different way of estimating the spectrum of a signal, which is typically less noisy.

(a) Power spectral density [6 P]

Instead of examining the frequency spectrum that results from the D(T)FT, one often instead looks at the **power spectral density (PSD)**. It shows how much of the signal's energy is concentrated on each frequency band.

Let $X(\omega)$ be the DFT of a signal $x \cdot w$ of length N, where w is a window function. Then, its PSD is defined as

$$\mathrm{PSD}\{x\}(\omega) = \frac{2|X(\omega) \cdot \mathbb{1}_{\{\omega \geq 0\}}(\omega)|^2}{f_s \cdot S}, \text{ where } \; S = \sum_{i=1}^{N} w[n]^2,$$

f_s is the sampling frequency and $\mathbb{1}_A$ is the characteristic function, i.e.,

$$\mathbb{1}_A(x) = \begin{cases} 1 \text{ if } x \in A \\ 0 \text{ otherwise.} \end{cases}$$

Implement a function `psd(x, w, fs)` to calculate the PSD from a signal x and window function w of the same length (real-valued numpy arrays) and the sampling frequency fs (float). You may use `np.fft.fft` to compute the DFT.

(b) The Slepian sequences [5 P]

The idea of multitapering is to apply mutually orthogonal windows (tapers) to the signal and average their PSDs. Typically, the Slepian sequences, also called discrete prolate spheroidal sequences (DPSS), are used.

- Use `scipy.signal.windows.dpss` to create the first three tapers of length $M = 500$ (use $NW = 1$).
- Plot all three tapers in one plot. Make sure to include a legend showing the labels of the different lines.
- Verify computationally that the tapers are indeed pairwise orthogonal to each other. Explain your computation in a short comment.

(c) Multitaper spectrum [5 P]

The multitaper spectrum is computed by applying the tapers to the signal and computing the PSD for each of these signals separately. Then, these different estimations are averaged. Implement a function `multitaper(x, fs, k)`, where x is the signal (real-valued numpy array of arbitrary length), fs is the sampling frequency in Hz and k is the number of tapers that should be used. Within the function, set the parameter NW of `scipy.signal.windows.dpss` to the length of the signal in seconds.

Hint: use the function `psd(x, w, fs)` from part (a) to compute the PSD of each windowed signal.

(d) Testing the function [5 P]

- Import the audio file `speech.wav` using `scipy.io.wavfile.read`.
- Print the signal's sampling frequency.
- Plot the signal in the time and frequency domain (e.g., using the functions defined in exercise 1). You may use `np.fft.fft` to compute the DFT.
- Compute and plot the PSD and the multitaper spectrum using 20 tapers.
- Describe your observations in a comment.

3 The short-time Fourier transform (STFT) [17 P]

The short-time Fourier transform (STFT) offers a unified view of both the frequency components of a signal and its development over time. The signal is cut into shorter windows, and the spectrum is calculated for each window separately.

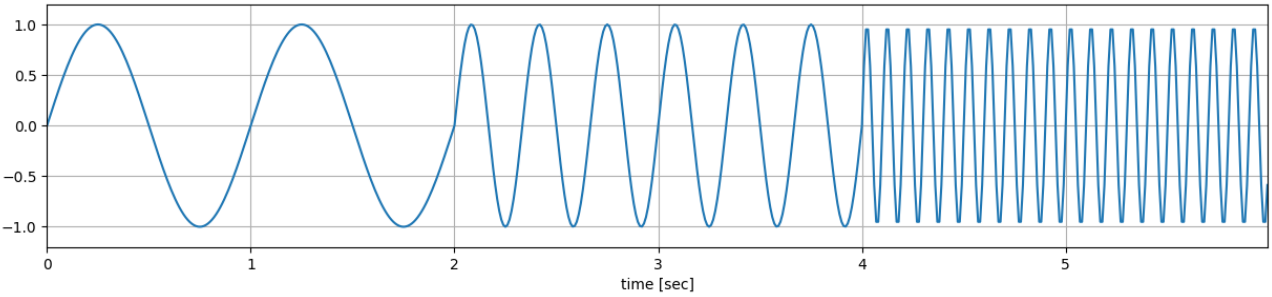
(a) STFT implementation [6 P]

Implement a function `stft(x, fs, wlen, win_func=False)` computing the STFT of a given signal x (numpy array of arbitrary length), fs is the sampling frequency (float) and wlen is the window length in seconds (float). The function should compute the DFT (`numpy.fft.fft`) for each segment of length wlen separately (no overlap) and return a complex-valued array of the shape `(n_windows, n_frequencies)`. When the boolean parameter `win_func` is set to `True`, multiply each segment with the *Hann* window function.

Hint: Use scipy.signal.windows.hann to create the Hann window function.

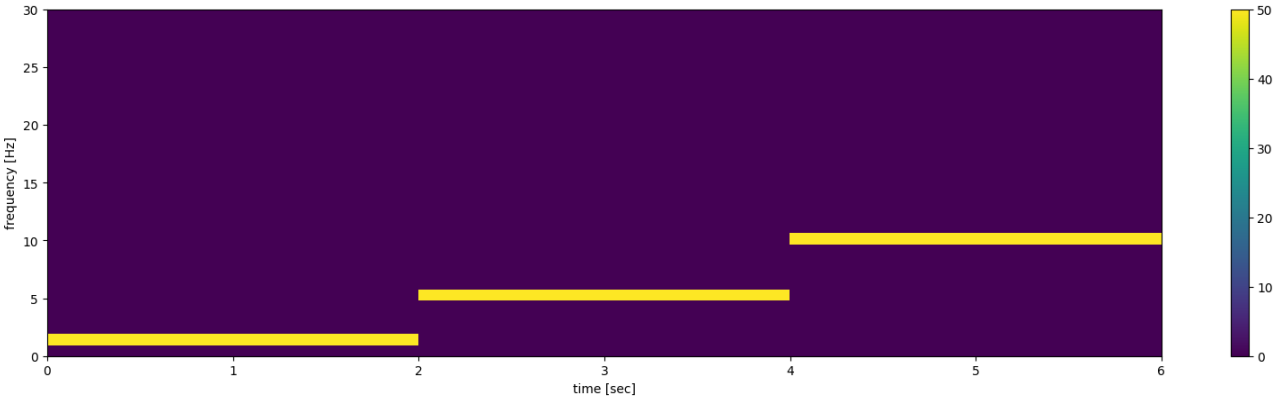
(b) Plotting the STFT [7 P]

- Define the signal shown below with a sampling frequency of 100Hz.



- Compute the signal's STFT using your previously defined function, using a window size of 1 second.
- Define a function `plot_stft(x_stft, fs, fmax=None, log=False, colormap_min=None, colormap_max=None)` that plots the amplitude of the STFT as an image. The function should:
  - Plot only the **positive frequencies**. If `fmax` is specified, limit the plot to frequencies up to `fmax`.
  - Plot the two amplitude spectra on a logarithmic scale (log-scaled, `numpy.log`) if `log=True`.
  - Use `plt.imshow` to plot, with:
    - `extent` make sure the axis ticks are labeled correctly in seconds and Hz,
    - `vmin=colormap_min` and `vmax=colormap_max` optionally adjust the color scale,
    - and use `aspect='auto'` and `origin='lower'`.
  - Add a **colorbar** to the plot.

- Plot the STFT of the above signal with `fmax=30` and default parameters. The result should look like this:



(c) Real-world example [4 P]

- For the signal `speech.wav` from exercise 2 (c), compute the STFT with a window length of 0.5 seconds.
- Plot the STFT with a suitable `fmax`.
- There are two different speakers. One has a higher voice than the other. Can you tell them apart without listening to the file? If so, who is speaking when, and how do you know? Write your observations in a comment.

4 Noise Gating via STFT [27 P]

In this task, you will implement a denoising method called *Noise Gating*. This method removes background noise in a signal by attenuating the signal below a certain threshold. In our case, signal parts below the threshold will be set to zero, and the parts above the threshold will pass through. The signal will be compared to the threshold in the frequency domain, eliminating unwanted frequencies below the threshold. You can find a more detailed introduction under `https://en.wikipedia.org/wiki/Noise_gate`.

(a) Inspection of the noisy audio signal [2 P]:

- Load and play the noisy audio file *noisy.wav* with `sounddevice.play()`.
- Plot the audio signal in the time domain, and display the x-axis in *seconds*.

(b) STFT of the audio signal [5 P]:

Compute the STFT of the audio signal and plot the amplitude spectrum of each segment of your output **W** with:

- the estimated windows (`win_func=False`), and
- the *Hann* windows (`win_func=True`).

Use your functions `stft` and `plot_stft` from excercise 3

Plot the two amplitude spectra on a logarithmic scale (`log=True`).

- Discuss the effects of the different windowing methods.
- Define the length of the segments with `win_len=0.2`.
- Use the same color scale for both amplitude spectra to enable comparison. Set `colormap_min` to 0 and `colormap_max` to the maximum amplitude value across both methods.
- What is the frequency resolution of your segments?

(c) Clean your audio spectrum with noise gaiting [12 P]:

1. Write a function

   $$\mathbf{W}\texttt{\_clean = noise\_gate(}\mathbf{W}\texttt{,t\_noise,c\_thresh)}$$

   which takes the noisy matrix **W**, the length of the noise epoch `t_noise` in seconds, and a factor for calculating the threshold of the gate `c_thresh` as input parameters. The output matrix **W**`_clean` represents the cleaned output matrix and has the same dimension as **W**.

   - The first five seconds of the audio signal contain only true background noise. Define the length of that epoch (`t_noise=5`), cut the epoch out of your noisy matrix **W**, and use that epoch to calculate the threshold of your noise gate. **Hint:** Consider the sampling frequency and the indices of your segments to find the cut-off for your epoch.
   - Calculate the threshold of your noise gate with $\texttt{thresh} = \mu + (\texttt{c\_thresh} \cdot \sigma)$

- μ: The mean of the log-scaled amplitudes at each frequency inside your noise epoch
- σ: The std of the log-scaled amplitudes at each frequency inside your noise epoch

Hint: The vectors μ and σ should have the same length as the number of frequencies in your matrix **W**.

- Compare the log-scaled amplitudes with the noise gate threshold `thresh` at every segment of your noisy matrix **W**. If an amplitude is below the threshold of their respective frequency, set that amplitude to zero.
- Return the clean matrix **W**`_clean`.

2. Apply your function `noise_gate()` with a threshold factor `c_thresh=3` on the output matrix **W** with
   - the estimated windows (`win_func=False`), and
   - the *Hann* windows (`win_func=True`).

   Plot the **log-scaled amplitude spectra of each segment of the cleaned output matrices** and discuss the effect of the cleaning process via noise gating.

(d) Reconstructing the cleaned audio signal [8 P]:

1. Implement a function `istft(X_stft)` that computes the inverse Short-Time Fourier Transform (iSTFT) of a given STFT matrix `X_stft`. Compute the Inverse Fourier Transform (`numpy.fft.ifft`) for each window. Return the reconstructed signal as a 1D numpy array in the time domain.
2. Apply your function `istft()` on the cleaned output matrix **W**`_clean` with
   - the estimated windows (`win_func=False`), and
   - the *Hann* windows (`win_func=True`).
3. To optimize your cleaned audio signal, you can scale it up to the maximum value (32767) in the 16-bit integer range:

   ```
   z_clean_scale = numpy.int16(z_clean/numpy.max(z_clean)*32767)
   ```

   Use the function `write()` from the library `scipy.io.wavfile` to save the cleaned audio files.
   - Play the cleaned audio signals and discuss the differences to the noisy audio signal.
   - Plot the cleaned audio signals in the time domain, and discuss the differences to the noisy audio signal.

5 Digital filter design and resampling [27 P]

In the first task, you will resample an audio signal. As shown in the lecture, when resampling (down- and/or upsampling) a signal, you also need to apply a low-pass filter. Therefore, We will first guide you through the steps of designing and applying a finite impulse response (FIR) filter via windowing:

1. Analytically specify the desired frequency response
2. Apply the inverse Fourier Transform to obtain the impulse response
3. Use a window function to obtain a finite-length filter

After that, you will apply the filter to an audio signal to perform a proper resampling.

(a) Design an ideal zero-phase FIR low-pass filter [10 P]

1. Write a function

   ```
   H_0, h_0 = ideal_lowpass(fc, fs, M)
   ```

   which takes the cut-off frequency `fc` in Hz, the sampling frequency `fs` in Hz, and the length of the filter `M` in samples as input parameters. The output `H_0` is the amplitude spectrum of the ideal low-pass filter in the frequency domain, and the output `h_0` is the impulse response in the time domain.

   - Specify the amplitude spectrum of your filter such that

     $$H_0(f) = \begin{cases} 1, |f| \leq f_c \\ 0, \text{otherwise} \end{cases}$$

     Hints:

     - Consider the frequency resolution of your filter: $f_{\text{res}} = \frac{f_s}{M}$.
     - Since you are designing a zero-phase filter, you don't have to specify a phase.

   - Apply the inverse Fourier Transform to $H_0(f)$ to obtain the impulse response $h_0[n]$.

     Hint: Use the functions `fftshift` and `ifft` from the library `numpy.fft`.

2. Design a *Hann* window, by writing another function `hann_window(M)` which takes the length of the window `M` in samples as an input parameter and returns the window `w_hann` in the time domain. The *Hann* window is defined as follows:

   $$w_{\text{hann}}[n] = \begin{cases} 0.5 - 0.5\cos(\frac{2\pi \cdot n}{M}), 0 \leq n \leq M-1 \\ 0, \text{otherwise} \end{cases}$$

3. Write a function to obtain an FIR filter

   ```
   H_fir, h_fir = fir_lowpass(fc, fs, M)
   ```

   which takes the cut-off frequency `fc` in Hz, the sampling frequency `fs` in Hz, and the length of the filter `M` in samples as input parameters. The output `H_fir` is the amplitude spectrum of the FIR low-pass filter in the frequency domain, and the output `h_fir` is its impulse response in the time domain.

   - Call your functions `ideal_lowpass` and `hann_window`.
   - Multiply your window function $w_{\text{hann}}[n]$ with the impulse response of your filter $h_0[n]$ to obtain a finite-length (FIR) filter:

$$h_{\text{fir}}[n] = h_0[n] \cdot w_{\text{hann}}$$

- Apply the Fourier Transform to $h_{\text{fir}}[n]$ to obtain the amplitude spectrum $H_{\text{fir}}(\omega)$.

  Hint: Use the functions `fftshift` and `fft` from the library `numpy.fft`.

4. Define a filter of length $M = 501$, a cut-off frequency $f_c = 30$ Hz, and a sampling frequency $f_s = 1000$ Hz.

   - Plot your filter in the time and the frequency domain **before and after** you apply your window function.
   - Which differences can you find between before and after applying your window function (in the time and frequency domain)?

(b) Test your filter on a signal with multiple frequencies [5 P]

1. Define a signal

$$x[n] = \sum_{k \in \{5,20,50,100\}} \sin(2\pi \frac{k}{f_s} n)$$

   with length $N = 2000$ ($n \in [0, N-1]$) and sampling frequency $f_s = 1000$ Hz. Plot the signal in the time and frequency domain.

2. Apply your finite-length filter $h_{\text{fir}}[n]$ to your signal $x[n]$ by using convolution:

$$x_{\text{filt}}[n] = x[n] * h_{\text{fir}}[n]$$

   Hint: Use the function `numpy.convolve` with the parameter `mode='same'` .

   - Plot the filtered signal $x_{\text{filt}}[n]$ in the time and frequency domain.
   - Does your filter work properly? Discuss your results!

(c) Downsample the audio signal [12 P]

1. Inspection of the audio signal:
   - Use `scipy.io.wavfile.read` to load the audio file *beethoven.wav*.
   - Print the sampling rate and the audio signal's length (in samples and seconds).
   - The audio file contains a stereo signal; convert it to a mono signal.
   - Use `sounddevice.play()` to play the audio signal.
   - Plot the signal in the time and frequency domain.
2. Naively downsample the audio signal by the factor $c_{\text{down}} \in \{5, 10, 20\}$.
   - Take every c_{down}^{th} sample of your audio signal.

     Example: $c_{\text{down}} = 2$, $x[n] = [1, 2, 3, 4, 5, 6] \rightarrow x_{\text{down}}[n] = [1, 3, 5]$
   - Update the sampling frequency $f_{s_{\text{down}}} = \frac{f_s}{c_{\text{down}}}$

- Print the sampling rate and length (in samples and seconds) of the naively downsampled signals.
- Play the naively downsampled audio signals and compare them to the original audio signal.

  Hint: Divide your downsampled signal by a factor of $clip = \frac{f_s}{2}$ to prevent your signal from clipping.

3. Properly downsample your audio signal by the factor $c_{\text{down}} \in \{5, 10, 20\}$ by applying an FIR filter
   - Use the same filter length as in (b) and the sampling frequency of your audio signal.
   - Calculate the cut-off frequency $f_c = \frac{\frac{f_s}{2}}{c_{\text{down}}}$
   - Apply the FIR filter to your audio signal and then take every c_{down}^{th} sample of your audio signal.
   - Play the properly downsampled audio signals and compare them to the naively downsampled signals and the original audio signal.
   - Print the cut-off frequency and the length (in samples and seconds) of the properly downsampled signals.
4. Compare the amplitude spectrum of the original audio signal with the spectra of the naively and the properly downsampled signals for all three downsampling factors c_{down}.
   - Plot all amplitude spectra in a range of $[-\frac{f_{s_{\text{down}}}}{2}, \frac{f_{s_{\text{down}}}}{2}]$, with respect to the corresponding downsampling factor c_{down}.
   - What differences do you see between the original spectrum, the naively downsampled spectrum, and the properly downsampled spectrum and
   - What differences do you see between the different downsampling factors c_{down}?